

Institut Supérieur des Arts Multimédia de la Manouba
ISAMM — Licence Informatique Multimédia

Cours Synthétique

Introduction au Langage VHDL

VHSIC Hardware Description Language

Réalisé par : Bargaoui Ahmed | Groupe : 2IM4 | Module : Circuit logique programmable | 2025-2026

Sommaire

1. Introduction au VHDL	3
1.1 Qu'est ce que le VHDL ?	3
1.2 Historique	3
1.3 Domaines d'application	3
2. Les Concepts Fondamentaux	4
2.1 L'Entité (ENTITY)	4
2.2 L'Architecture (ARCHITECTURE)	5
2.3 Les Signaux et les Ports	5
2.4 Le PROCESS	6
3. Exemples de Code Commentés	7
3.1 Exemple 1 : Porte logique AND	7
3.2 Exemple 2 : Bascule D (registre simple)	8
4. Conclusion et Points Clés	9

1. Introduction au VHDL

1.1 Qu'est ce que le VHDL ?

Le **VHDL** (qui veut dire **VHSIC Hardware Description Language**, et VHSIC c'est **Very High Speed Integrated Circuit**) c'est un langage de description materielle. Autrement dit, c'est pas un langage de programmation classique comme le C ou Python, mais plutôt un langage qui permet de **décrire le comportement et la structure d'un circuit électronique numérique**.

En gros, quand on veut concevoir un circuit (genre une porte logique, un compteur, ou meme un processeur entier), on peut écrire son fonctionnement en VHDL, et ensuite un logiciel (comme Vivado ou Quartus) va transformer ca en vrai circuit physique sur un FPGA ou un ASIC.

💡 *FPGA = Field Programmable Gate Array → c'est un composant électronique qu'on peut reconfigurer à souhait grâce à un fichier VHDL compilé.*

1.2 Historique (en bref)

Le VHDL a été crée dans les années 1980 par le Departement de la Défense américain (DoD) pour standardiser la documentation des circuits integres. En 1987, il est devenu un standart IEEE (IEEE 1076). Depuis, il a eu plusieurs mises à jours (1993, 2000, 2008) pour ajouter des nouvelles fonctionnalités.

- 1983 : Début du développement par le DoD
- 1987 : Standardisation IEEE 1076-1987
- 1993 : Révision majeure (VHDL-93)
- 2008 : Dernière grande version (VHDL-2008)

1.3 Domaines d'application

Le VHDL est utilisé dans beaucoup de domaines :

- Conception de processeurs et microcontrôleurs
- Systèmes embarqués et temps réel
- Télécommunications (encodeurs, décodeurs...)
- Aérospatiale et défense
- Electronique grand public (TV, routeurs...)

2. Les Concepts Fondamentaux

2.1 L'Entité (ENTITY)

L'entité c'est comme la **"boîte noire"** de notre composant. Elle définit juste les entrées et les sorties, sans expliquer comment ça marche à l'intérieur. On peut la comparer à un schéma de branchement.

La syntaxe de base d'une entité c'est :

```
ENTITY nom_du_composant IS
  PORT (
    -- Déclaration des ports
    entreel : IN  STD_LOGIC;
    entree2 : IN  STD_LOGIC;
    sortiel : OUT STD_LOGIC
  );
END ENTITY nom_du_composant;
```

Les types de ports les plus courants :

•

Mode	Sens	Utilisation
IN	Entrée	Signal reçu de l'extérieur (lecture seule)
OUT	Sortie	Signal envoyé vers l'extérieur (écriture seule)
INOUT	Bidirectionnel	Peut être lu ET écrit (bus par exemple)
BUFFER	Sortie retroactive	Sortie qu'on peut aussi lire en interne

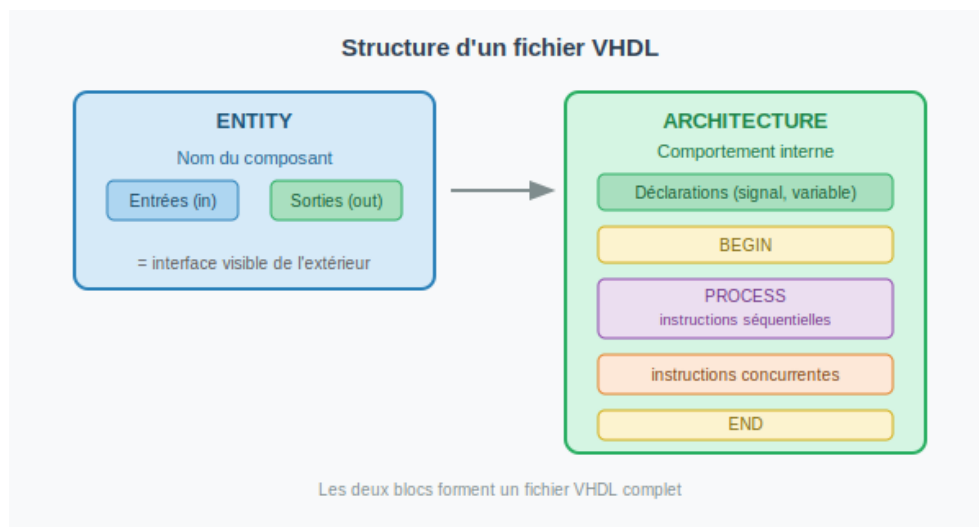


Figure 1 – Structure générale d'un fichier VHDL (entité + architecture)

2.2 L'Architecture (ARCHITECTURE)

L'architecture c'est la **partie interne** du composant, là où on décrit vraiment ce que fait le circuit. Elle est toujours associée à une entité précise.

La syntaxe est la suivante :

```
ARCHITECTURE nom_archi OF nom_composant IS
  -- Zone de déclarations (signaux internes, constantes...)
  SIGNAL sig_interne : STD_LOGIC;
BEGIN
  -- Zone de descriptions (instructions)
  sig_interne <= entreel AND entree2;  -- exemple
END ARCHITECTURE nom_archi;
```

2.3 Les Signaux et les Ports

En VHDL, on a deux types de " **fils** " pour transporter de l'information :

Les Ports : ce sont les connections externes du composant (définies dans l'ENTITY). On les voit de l'extérieur.

Les Signaux (SIGNAL) : ce sont des fils internes a l'architecture. On les voit pas de l'extérieur.

 *Attention ! En VHDL, l'affectation d'un signal se fait avec <= et non pas = comme en C. Par exemple : sortie <= entree1 AND entree2;*

Le type le plus utilisé pour les signaux logiques c'est **STD_LOGIC** qui peut prendre plusieurs valeurs : '0', '1', 'Z' (haute impédance), '-' (don't care), etc. Pour des bus, on utilise

STD_LOGIC_VECTOR.

```
SIGNAL compteur : STD_LOGIC_VECTOR(7 DOWNTO 0); -- bus 8 bits
SIGNAL drapeau : STD_LOGIC; -- 1 bit
```

2.4 Le PROCESS

Le **PROCESS** c'est une des notions les plus importantes en VHDL. Il permet d'ecrire des instructions **séquentielles** (c'est a dire qui s'executent dans l'ordre, un peu comme en C). En dehors d'un process, toutes les instructions VHDL s'exécutent **en parallèle** (ce qui est tres différent de la programmation classique !).

La syntaxe d'un PROCESS :

```
PROCESS (liste_de_sensibilite)
  -- liste_de_sensibilite : les signaux qui declenchent le process
BEGIN
  -- instructions séquentielles ici
  IF condition THEN
    signal_out <= '1';
  ELSE
    signal_out <= '0';
  END IF;
END PROCESS;
```

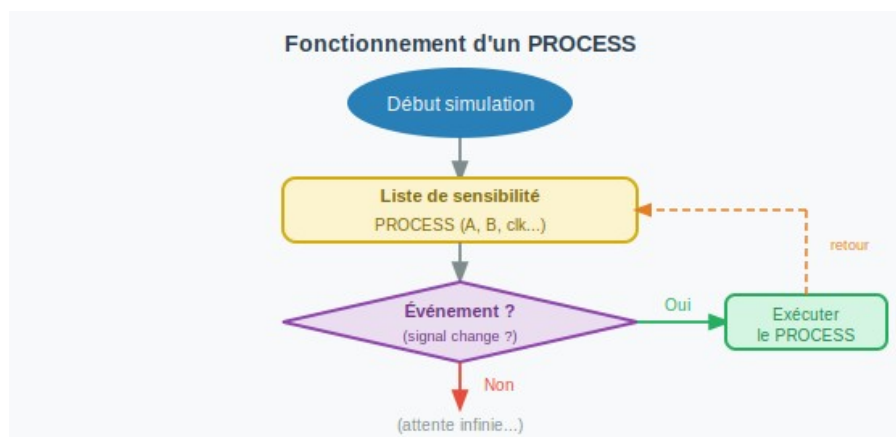


Figure 2 – Fonctionnement d'un PROCESS et sa liste de sensibilité

💡 Un PROCESS se déclenche à chaque fois qu'un des signaux de sa liste de sensibilité change de valeur. Si la liste est vide, le process tourne en boucle infinie (utile pour les testbenchs).

3. Exemples de Code Commentés

3.1 Exemple 1 : Porte logique AND (2 entrées)

Voici le code VHDL le plus simple possible : une porte AND avec deux entrees A et B, et une sortie S.

```
-- Inclusion de la bibliotheque standard
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

-- Declaration de l'entite (interface)
ENTITY porte_and IS
  PORT (
    A : IN  STD_LOGIC;  -- 1ere entree
    B : IN  STD_LOGIC;  -- 2eme entree
    S : OUT STD_LOGIC    -- sortie (pas de ; sur le dernier port !)
  );
END ENTITY porte_and;

-- Description du comportement (architecture)
ARCHITECTURE comportement OF porte_and IS
BEGIN
  -- Instruction concurrente : S prend la valeur de A AND B
  -- Cette ligne s'execute tout le temps, pas besoin de PROCESS
  S <= A AND B;
END ARCHITECTURE comportement;
```

Et le chronogramme correspondant (simulation) :

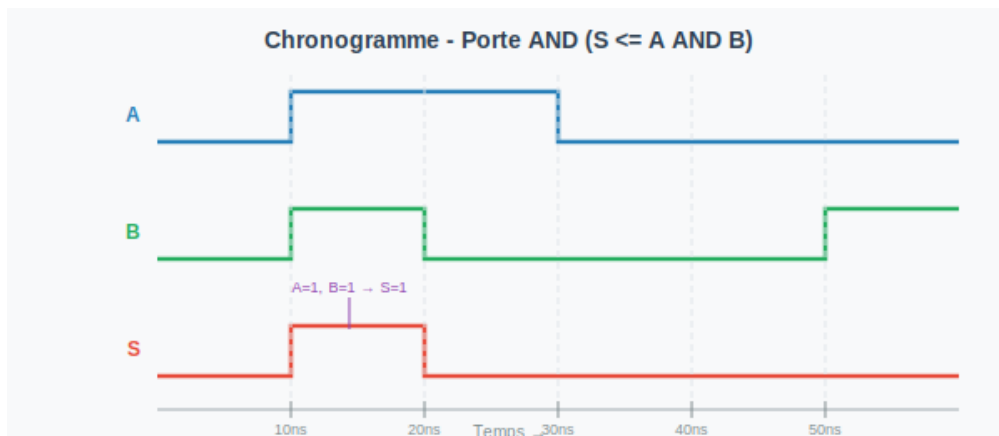


Figure 3 – Chronogramme de la porte AND : $S = A \text{ AND } B$

Explication du chronogramme : On voit que la sortie S passe à '1' uniquement quand A=1 ET B=1 en même temps. C'est exactement le comportement attendu d'une porte AND. Les changements de S suivent immédiatement les changements de A et B (sans délai dans cet exemple).

3.2 Exemple 2 : Bascule D (registre 1 bit)

La bascule D c'est un élément de mémoire très utilisé dans les circuits séquentiels. Elle capture la valeur de son entrée D au moment d'un front montant de l'horloge (clk).

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY bascule_d IS
  PORT (
    clk : IN  STD_LOGIC;  -- signal d'horloge
    rst : IN  STD_LOGIC;  -- reset asynchrone
    D   : IN  STD_LOGIC;  -- donnee en entree
    Q   : OUT STD_LOGIC   -- sortie
  );
END ENTITY bascule_d;

ARCHITECTURE sequentielle OF bascule_d IS
BEGIN
  PROCESS (clk, rst) -- le process reagit sur clk ou rst
  BEGIN
    -- Reset asynchrone : prioritaire sur tout
    IF rst = '1' THEN
      Q <= '0';
    -- Detection du front montant de l'horloge
    ELSIF rising_edge(clk) THEN
      Q <= D;      -- on capture D
    END IF;
  END PROCESS;
END ARCHITECTURE sequentielle;
```

Points importants de cet exemple :

-
- **rising_edge(clk)** : cette fonction detecte le front montant de l'horloge (quand clk passe de '0' a '1'). C'est la facon standard en VHDL de faire du sequentiel synchrone.
- **rst asynchrone** : il est dans la liste de sensibilite donc le reset peut agir meme sans front d'horloge.
- L'ordre des IF/ELSIF defini les priorités (rst est prioritaire sur clk).

💡 A retenir : tout ce qui est dans un **PROCESS** avec **rising_edge()** c'est du logique sequentiel (memoire). Tout ce qui est dehors c'est du combinatoire (instantané).

4. Conclusion et Points Clés

Le VHDL est un langage puissant mais qui demande un peu de temps pour bien comprendre sa logique, surtout si on vient de la programmation classique. La différence principale c'est que en VHDL on décrit du matériel, donc toutes les instructions (hors PROCESS) s'exécutent en même temps, en parallèle.

Voici un résumé des points clés à retenir :

- **ENTITY** : définit les entrées/sorties du composant. C'est son interface avec l'extérieur.
- **ARCHITECTURE** : décrit le comportement interne. Toujours associée à une ENTITY.
- **SIGNAL** : des fils internes pour transporter l'information entre les différentes parties.
- **PROCESS** : zone d'instructions séquentielles, déclenchée par la liste de sensibilité.
- **<=** : opérateur d'affectation pour les signaux (et pas = comme en C !).
- **STD_LOGIC** : le type standard pour les signaux logiques (nécessite IEEE.STD_LOGIC_1164).
- **rising_edge()** : fonction pour détecter les fronts montants d'horloge dans les circuits séquentiels.

Ce qu'on a vu vs ce qu'on a pas encore vu : ce cours c'est vraiment les bases. Il reste beaucoup de choses à apprendre comme les machines d'états (FSM), la généricité (GENERIC), l'instanciation de composants, les packages, les fonctions et procédures, et aussi les testbenches pour simuler et vérifier le bon fonctionnement d'un circuit.

Conseil final : le meilleur moyen d'apprendre le VHDL c'est de pratiquer ! On vous conseille de tester vos codes avec des simulateurs gratuits comme GHDL + GTKWave, ou avec les logiciels des fabricants de FPGAs (Vivado pour Xilinx, Quartus pour Intel/Altera).

— Fin du document —

Cours réalisé à ISAMM — Licence Informatique Multimédia | 2024-2025